M5_Regressió_Logística_Multiclasse

January 12, 2026

1 Regressió Logística

1.1 Detecció de Overfitting i "ajustament" del model

L'overfitting en aquest tipus de 'classificadors' acostuma a ser menys comú però veguem com podem detectar-ho.

De la mateixa manera que ens altres models, en de calcular la diferència entre l'error comès amb el train i el test (diferència entre l'*accuracy* de train i l'*accuracy* del test).

Gap Train-Test	Interpretació
0-2%	Model amb comportament ideal
2-5%	Model amb comportament acceptable
>5%	Indica Overfitting

[ ]:
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay,␣
 ↪classification_report

# Carreguem les dades
breast_cancer = load_breast_cancer()
df = pd.DataFrame(breast_cancer.data, columns=breast_cancer.feature_names)
df['target'] = breast_cancer.target

X = df.drop('target', axis=1)
y = df['target'].values

# Train / Test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

[ ]:
```
X_train.head()
```

[ ]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	\
68	9.029	17.33	58.79	250.5	0.10660	
181	21.090	26.57	142.70	1311.0	0.11410	
63	9.173	13.86	59.20	260.9	0.07721	
248	10.650	25.22	68.01	347.0	0.09657	
60	10.170	14.88	64.55	311.9	0.11340	

	mean compactness	mean concavity	mean concave points	mean symmetry	\
68	0.14130	0.31300	0.04375	0.2111	
181	0.28320	0.24870	0.14960	0.2395	
63	0.08751	0.05988	0.02180	0.2341	
248	0.07234	0.02379	0.01615	0.1897	
60	0.08061	0.01084	0.01290	0.2743	

	mean fractal dimension	…	worst radius	worst texture	\
68	0.08046	…	10.31	22.65	
181	0.07398	…	26.68	33.48	
63	0.06963	…	10.01	19.23	
248	0.06329	…	12.25	35.19	
60	0.06960	…	11.02	17.45	

	worst perimeter	worst area	worst smoothness	worst compactness	\
68	65.50	324.7	0.14820	0.43650	
181	176.50	2089.0	0.14910	0.75840	
63	65.59	310.1	0.09836	0.16780	
248	77.98	455.7	0.14990	0.13980	
60	69.86	368.6	0.12750	0.09866	

	worst concavity	worst concave points	worst symmetry	\
68	1.25200	0.17500	0.4228	
181	0.67800	0.29030	0.4098	
63	0.13970	0.05087	0.3282	
248	0.11250	0.06136	0.3409	
60	0.02168	0.02579	0.3557	

	worst fractal dimension
68	0.11750
181	0.12840
63	0.08490
248	0.08147
60	0.08020

[5 rows x 30 columns]

Per millorar el comportament del model apliquem un escalat a les dades perquè tots els *features* tinguin la mateixa escala.

[ ]:
```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

## Línies de codi només per a la conversió a un DataFrame
df_train_scaled = pd.DataFrame(X_train, columns=X.columns)
df_train_scaled.head()
```

[ ]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	\
0	-1.440753	-0.435319	-1.362085	-1.139118	0.780573	
1	1.974096	1.733026	2.091672	1.851973	1.319843	
2	-1.399982	-1.249622	-1.345209	-1.109785	-1.332645	
3	-0.981797	1.416222	-0.982587	-0.866944	0.059390	
4	-1.117700	-1.010259	-1.125002	-0.965942	1.269511	

	mean compactness	mean concavity	mean concave points	mean symmetry	\
0	0.718921	2.823135	-0.119150	1.092662	
1	3.426275	2.013112	2.665032	2.127004	
2	-0.307355	-0.365558	-0.696502	1.930333	
3	-0.596788	-0.820203	-0.845115	0.313264	
4	-0.439002	-0.983341	-0.930600	3.394436	

	mean fractal dimension	…	worst radius	worst texture	worst perimeter	\
0	2.458173	…	-1.232861	-0.476309	-1.247920	
1	1.558396	…	2.173314	1.311279	2.081617	
2	0.954379	…	-1.295284	-1.040811	-1.245220	
3	0.074041	…	-0.829197	1.593530	-0.873572	
4	0.950213	…	-1.085129	-1.334616	-1.117138	

	worst area	worst smoothness	worst compactness	worst concavity	\
0	-0.973968	0.722894	1.186732	4.672828	
1	2.137405	0.761928	3.265601	1.928621	
2	-0.999715	-1.438693	-0.548564	-0.644911	
3	-0.742947	0.796624	-0.729392	-0.774950	
4	-0.896549	-0.174876	-0.995079	-1.209146	

	worst concave points	worst symmetry	worst fractal dimension
0	0.932012	2.097242	1.886450
1	2.698947	1.891161	2.497838
2	-0.970239	0.597602	0.057894
3	-0.809483	0.798928	-0.134497
4	-1.354582	1.033544	-0.205732

[5 rows x 30 columns]

Calculem el *gap* entre l'*accuracy* entre train i test:

[ ]:
```
model = LogisticRegression(solver='liblinear')
model.fit(X_train, y_train)

train_acc = model.score(X_train, y_train)
test_acc = model.score(X_test, y_test)
gap = (train_acc - test_acc) * 100

print(f"gap = {train_acc:.3f} - {test_acc:.3f} = {gap:.1f}")
```

```
gap = 0.987 - 0.974 = 1.3
```

Mireu que passa si no realitzem aquest escalat (torna a executar el codi però comentant la part de l'escalat).

Regularització

Encara disposem d'un altre factor per acabar d'ajustar el model, per intentar a obtenir un millor resultat.

El paràmetre C permet "ajustar" els coeficients per tenir un millor resultat.

[ ]:
```
C_values = [0.01, 0.1, 1, 10, 100]

print("C".rjust(8) + "Train".rjust(8) + "Test".rjust(8) + "Gap%".rjust(8))
print("-"*32)

for C in C_values:
    model = LogisticRegression(C=C, solver='liblinear')
    model.fit(X_train, y_train)

    train_acc = model.score(X_train, y_train)
    test_acc = model.score(X_test, y_test)
    gap = (train_acc - test_acc) * 100

    print(f"{C:>8.2f}{train_acc:>8.3f}{test_acc:>8.3f}{gap:>8.2f}")
```

```
       C   Train    Test    Gap%
--------------------------------
    0.01   0.967   0.982   -1.54
    0.10   0.982   0.991   -0.88
    1.00   0.987   0.974    1.31
   10.00   0.989   0.974    1.53
  100.00   0.993   0.939    5.48
```

Quin serà el millor valor de C?

La regularització afegeix un "castig" als coeficients massa grans per evitar que el model "memoritzi" les dades d'entrenament.
Hi ha dos tipus principals: L1 (Lasso) que posa alguns coeficients a zero (elimina features inútils) i L2 (Ridge) que els fa tots més petits però no els elimina.
El paràmetre C decideix quant de "castig" s'aplica.

[ ]:
```
models = {
    'No Regularització': LogisticRegression(C=0.1, solver='liblinear'),
    'L1 (Lasso)': LogisticRegression(penalty='l1', C=0.1, solver='liblinear'),
    'L2 (Ridge)': LogisticRegression(penalty='l2', C=0.1, solver='lbfgs'),
}
```

[ ]:
```
print("Nom".rjust(20) + "Train".rjust(8) + "Test".rjust(8) + "Gap%".rjust(8))
print("-"*45)

for name, model in models.items():
  model.fit(X_train,y_train)
  y_pred = model.predict(X_test)

  train_acc = model.score(X_train, y_train)
  test_acc = model.score(X_test, y_test)
  gap = (train_acc - test_acc) * 100

  print(f"{name:>20}{train_acc:>8.3f}{test_acc:>8.3f}{gap:>8.2f}")
```

```
                 Nom   Train    Test    Gap%
---------------------------------------------
   No Regularització   0.982   0.991   -0.88
         L1 (Lasso)   0.980   0.965    1.53
         L2 (Ridge)   0.980   0.982   -0.22
```

Per obtenir el millor model hauriem d'ajustar els valors de C i aplicar les diferents regularitzacions.

2 Regressió Logística Multiclasse

Classificació multiclasse: anomemen classificació multiclasse quan disposem de tres o més classes de sortides entre les quals triar, a diferència de la classificació binària (només dues opcions).

Veurem l'exemple amb dades per a reconeixement de dígits manuscrits:

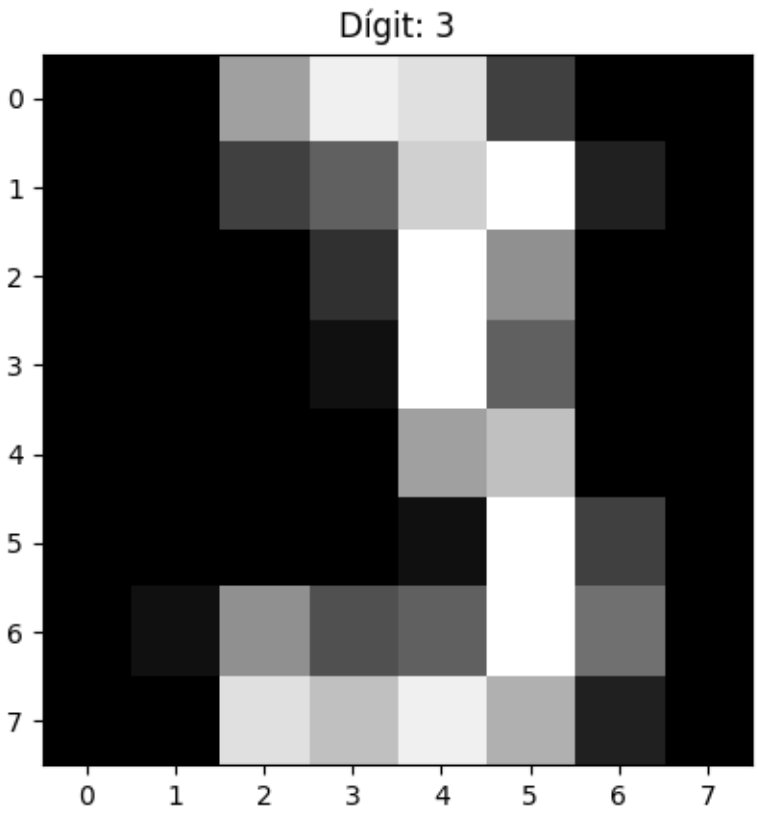
[ ]:
```
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler

digits = load_digits()
df = pd.DataFrame(digits.data, columns=digits.feature_names)
df['target'] = digits.target

df.head()
```

[ ]:

	pixel_0_0	pixel_0_1	pixel_0_2	pixel_0_3	pixel_0_4	pixel_0_5	\
0	0.0	0.0	5.0	13.0	9.0	1.0	
1	0.0	0.0	0.0	12.0	13.0	5.0	
2	0.0	0.0	0.0	4.0	15.0	12.0	

```
3        0.0        0.0        7.0       15.0       13.0        1.0
4        0.0        0.0        0.0        1.0       11.0        0.0

   pixel_0_6  pixel_0_7  pixel_1_0  pixel_1_1  …  pixel_6_7  pixel_7_0  \
0        0.0        0.0        0.0        0.0  …        0.0        0.0
1        0.0        0.0        0.0        0.0  …        0.0        0.0
2        0.0        0.0        0.0        0.0  …        0.0        0.0
3        0.0        0.0        0.0        8.0  …        0.0        0.0
4        0.0        0.0        0.0        0.0  …        0.0        0.0

   pixel_7_1  pixel_7_2  pixel_7_3  pixel_7_4  pixel_7_5  pixel_7_6  \
0        0.0        6.0       13.0       10.0        0.0        0.0
1        0.0        0.0       11.0       16.0       10.0        0.0
2        0.0        0.0        3.0       11.0       16.0        9.0
3        0.0        7.0       13.0       13.0        9.0        0.0
4        0.0        0.0        2.0       16.0        4.0        0.0

   pixel_7_7  target
0        0.0       0
1        0.0       1
2        0.0       2
3        0.0       3
4        0.0       4

[5 rows x 65 columns]
```

[ ]:
```
plt.imshow(digits.images[60], cmap='gray')
plt.title(f'Dígit: {digits.target[60]}')
plt.show()
```



[ ]:
```
X = df.drop('target', axis=1)
y = df['target'].values
```

[ ]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size = 0.2, random_state = 42)
```

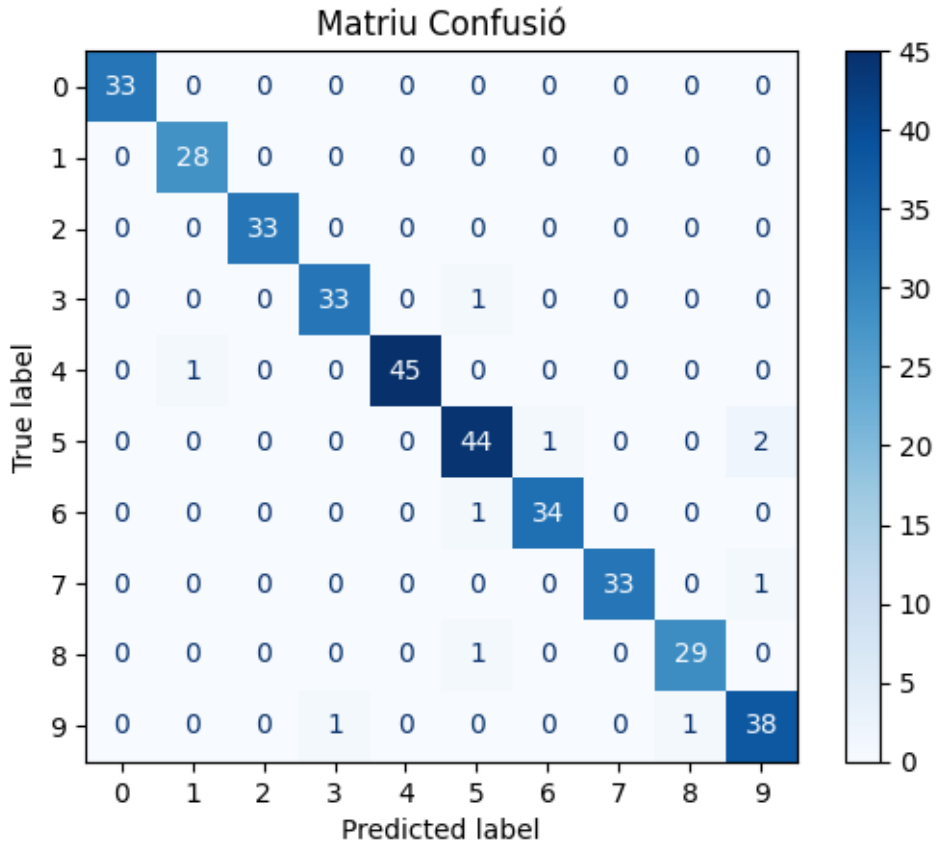
[ ]:
```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

[ ]:
```
model = LogisticRegression(solver='lbfgs', multi_class='multinomial')
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247:
FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed
in 1.7. From then on, it will always use 'multinomial'. Leave it to its default
value to avoid this warning.
  warnings.warn(
```

3-5 classes equilibrades → 'multinomial' + solver='lbfgs'
10+ classes o desequilibri → 'ovr' + solver='lbfgs'

[ ]:
```
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues')
plt.title('Matriu Confusió')
plt.show()
```



[ ]:
```
print(classification_report(y_test, y_pred,
                            target_names=[f'Digit {i}' for i in range(10)]))
```

```
              precision    recall  f1-score   support

     Digit 0       1.00      1.00      1.00        33
     Digit 1       0.97      1.00      0.98        28
     Digit 2       1.00      1.00      1.00        33
     Digit 3       0.97      0.97      0.97        34
     Digit 4       1.00      0.98      0.99        46
     Digit 5       0.94      0.94      0.94        47
     Digit 6       0.97      0.97      0.97        35
     Digit 7       1.00      0.97      0.99        34
     Digit 8       0.97      0.97      0.97        30
```

Digit 9	0.93	0.95	0.94	40
accuracy			0.97	360
macro avg	0.97	0.97	0.97	360
weighted avg	0.97	0.97	0.97	360